TECHNICAL REFERENCE DOCUMENT

EduConnect Backend

Architecture · Schema · API Reference · Code Flow · Interview Preparation

Framework	Language	Database	Security	Build Tool
Spring Boot 4.0.2	Java 21	MySQL + Hibernate (JOINED)	JWT + Spring Security	Maven 3.9.12

TABLE OF CONTENTS

01 Architecture Overview

02 Database Schema

03 Code Flow & Request Lifecycle

04 API Endpoints Reference

05 Security & Middleware

06 Design Patterns in Use

07 Suggested Improvements

08 Microservices Migration Plan

09 Interview Q&A — Core Java

10 Interview Q&A — Collections & Streams

11 Interview Q&A — Design Patterns

12 Interview Q&A — Spring Boot & JPA

13 Interview Q&A — System Design & Architecture

1 Architecture Overview

EduConnect is a **monolithic Spring Boot application** following a classic **layered (N-tier) architecture** augmented by the **Strategy** and **Factory** design patterns. Each layer has a single, well-defined responsibility, and inter-layer communication flows strictly top-down via dependency injection.

Presentation Layer — Controllers

REST endpoints annotated with `@RestController`. Responsible for HTTP mapping, request parsing, and delegating to the service layer. Does not contain business logic. Swagger/OpenAPI documentation is auto-generated via `springdoc-openapi`.

Factory / Strategy Layer — UserFactory, AssessmentFactory

An intermediary layer between Controllers and Services. Implements the **Factory + Strategy pattern** to dynamically select role-based or type-based logic at runtime without conditionals. `UserFactory` routes registration and login to the correct `UserAuthStrategy`. `AssessmentFactory` routes create/submit to `AssignmentStrategy` or `QuizStrategy`.

Service Layer — Contract Interfaces + Implementations

All business logic lives here. Services are defined as interfaces (e.g., `StudentService`) and implemented (e.g., `StudentServiceImpl`), adhering to the **Interface Segregation Principle**. Services are `@Transactional` at appropriate granularity and interact with repositories exclusively.

Repository Layer — Spring Data JPA

Thin interfaces extending `JpaRepository<T, UUID>`. Query methods use derived query syntax (e.g., `findByStudentUserId`) and JPQL for complex joins (`@Query`). No SQL is written manually except in Flyway migration scripts.

Persistence Layer — MySQL + Hibernate

JPA entities map to a MySQL schema using **JOINED table-per-subclass inheritance** for the `User` hierarchy. File content (BLOBs) is stored inline in MySQL via `@Lob / LONGBLOB` columns. Flyway manages schema versioning.

Cross-Cutting Concerns

`JwtFilter` (authentication), `SecurityConfig` (authorization), `GlobalExceptionHandler` (`@ControllerAdvice`), `logback.xml` (structured logging to rolling files), and `SwaggerConfig` (API docs) operate across all layers.

Technology Stack

Spring Boot 4.0.2 · Spring Security 6 · Spring Data JPA · Hibernate ORM · jjwt 0.13 · MySQL 8 · Flyway 12 · JAVE (FFmpeg wrapper) · Lombok · springdoc-openapi 3 · JavaMailSender

2 Database Schema

Inheritance Strategy: JOINED (Table-Per-Subclass)

The `User` entity is the root of a class hierarchy. Hibernate creates a **base table** (`users`) and a **separate subclass table** for each role. All four subclass tables (`admin` , `teacher` , `student` , `parent`) use their primary key column as both PK and FK to `users.user_id` , enabled by `@PrimaryKeyJoinColumn` .

> **Trade-off:** JOINED inheritance is space-efficient (no null columns) but every polymorphic query incurs a JOIN. For the current scale this is acceptable; at high read volume consider `SINGLE_TABLE` with discriminator columns.

Core Entity Relationships

Relationship	Cardinality	Notes
`Teacher → Course`	1:N	A teacher owns multiple courses; FK on `course.teacher_id`
`Course → CourseModule`	1:N (CascadeAll)	Modules deleted when course deleted; `@PrePersist` auto-assigns `sequence_order`
`Course → Assessment`	1:N	Each course has multiple assessments (Assignments or Quizzes)
`Assessment → Assignment/Quiz`	1:0..1 each	Polymorphic composition; discriminated by `AssessmentType` enum
`Student → Enrollment → Course`	M:N via pivot	Unique constraint on `(student_id, course_id)` ; stores progress and grade
`Student → Submission → Assessment`	M:N via pivot	Unique constraint prevents duplicate submissions per student per assessment
`Submission → AssignmentAttachment`	1:N	BLOB files stored inline; extends `Attachment` via JOINED inheritance
`Quiz → Question → QuestionOption`	1:N:N	Three-level hierarchy; `isCorrectOption` boolean drives auto-grading
`Student → StudentDemographics`	1:0..1 (@MapsId)	Shared PK pattern; demographic table uses student's UUID as its own PK
`Parent → Student`	1:N	Parent can have multiple children; verified via email token before linking
`User → RefreshToken`	1:0..1	Upsert pattern; old token replaced on each new login

`User → AuditLog`	1:N (CascadeAll)	LOGIN, LOGOUT, CREATE actions tracked with resource string and timestamp

Notable Schema Decisions

BLOB Storage in MySQL

Both `StudentDocument` and `Attachment` store raw file bytes in `LONGBLOB` columns with `@Lob @Basic(fetch = FetchType.LAZY)`. The `LAZY` fetch is critical — without it Hibernate would load multi-megabyte BLOBs into every entity query. Even with lazy loading, this pattern couples database size to media volume and should be migrated to object storage in production.

Unique Constraints as Business Rules

`Enrollment` enforces `UNIQUE(student_id, course_id)`. `Submission` enforces `UNIQUE(assessment_id, student_id)`. `StudentQuizQuestionResponse` enforces `UNIQUE(submission_id, question_id)`. These database-level constraints act as a last line of defense against duplicate business events.

UUID as Primary Key

All entities use `UUID` PKs generated via `GenerationType.UUID`. UUIDs eliminate sequential ID enumeration attacks but are larger (16 bytes vs 8 for `BIGINT`) and less cache-friendly for index scans. A `BINARY(16)` storage type (as used in the Flyway seed) is the MySQL-optimal representation.

3 Code Flow & Request Lifecycle

3.1 User Registration Flow

1. **HTTP POST /v1/auth/register** — `AuthController.register()` receives a raw `User` entity in the request body (email, password, role, fullName).
2. **UserFactory.executeSave()** — chains three `UserValidator` functions: `EMAIL_VALID`, `PASSWORD_LENGTH`, `PASSWORD_STRENGTH`. Each returns `Optional<String>` (error message or empty). The chain uses the **Chain of Responsibility** pattern. On failure, `IllegalArgumentException` is thrown.
3. **Strategy Selection** — `strategyList.stream().filter(s → s.supports(role)).findFirst()` picks the correct `UserAuthStrategy` implementation (Admin, Student, Teacher, or Parent).
4. **Strategy.save()** — e.g., `StudentAuthStrategy.save()` builds a `Student` entity via the Lombok `@SuperBuilder`, BCrypt-encodes the password with strength 12, and persists via `StudentRepo.save()`.
5. **Response** — `AuthController` returns a `UserResponseDTO` containing only the email. Password is never included in any response DTO.

3.2 Authentication (Login) Flow

1. **HTTP POST /v1/auth/login** — receives `UserRequestDTO` (email, password, role string).
2. **UserFactory.verify()** — builds a transient `User` object from the DTO and routes to the matching strategy's `verify()` default method.
3. **AuthenticationManager.authenticate()** — Spring Security's `DaoAuthenticationProvider` loads the user via `EduconnectUserDetailsService`, compares the BCrypt hash, and either returns an authenticated token or throws `BadCredentialsException`.
4. **Token Generation** — `JWTService.generateToken(email)` creates a signed HS256 JWT with 10-minute expiry. `RefreshTokenServiceImpl.createToken(userId)` upserts a 32-byte random Base64 refresh token with 24-hour expiry in the `refresh_token` table.
5. **Audit Logging** — `AuditLogService.createAudit(userId, Action.LOGIN, role)` persists an `AuditLog` record linked to the user entity.
6. **Response** — `UserResponseDTO` with `accessToken`, `refreshToken`, `uuid`, `email`, `role`, and `name`.

3.3 Authenticated Request Flow (JWT Validation)

1. **JwtFilter.doFilterInternal()** — extracts the `Bearer` token from the `Authorization` header. Exceptions during extraction (expired, malformed) are caught and logged at DEBUG level; the filter continues the chain without setting authentication.

2. **UserDetails Load** — if a username is extracted and no authentication exists in the `SecurityContextHolder`, `EduconnectUserDetailsService.loadUserByUsername(email)` fetches the full `User` entity and wraps it in `UserPrinciples`.

3. **Token Validation** — `JWTService.validateToken(token, userDetails)` checks the username match and expiry. On success, a `UsernamePasswordAuthenticationToken` is placed in the `SecurityContextHolder`.

4. **Authorization Check** — `SecurityConfig` role-based rules are evaluated. A `403` produces a structured JSON response from `CustomAccessDeniedHandler`.

5. **Controller Execution** — the controller extracts the authenticated user via `@AuthenticationPrincipal UserPrinciples` where role-specific entity casting is needed (e.g., `(Teacher) userPrinciple.getUser()`).

3.4 Assessment Create & Submit Flow

1. **POST /v1/api/assessment/create** — receives `CreateAssessmentRequestDTO`. Jackson uses `@JsonTypeInfo` with property `assessmentType` to deserialize into the correct subclass (`CreateAssignmentRequestDTO` or `CreateQuizRequestDTO`) before the controller method even executes.

2. **AssessmentFactory.createAssessment()** — streams through `AssessmentStrategy` implementations, filters on `supports(assessmentType)`, and delegates to the matched strategy.

3. **AssignmentStrategy/QuizStrategy.createAssessment()** — validates teacher ownership via `canCreateAssessment(teacher, course)` (comparing `getUserId()`). Builds and persists `Assessment` + the typed entity (`Assignment` or `Quiz` with nested `Question` / `QuestionOption` lists) in a single `@Transactional` boundary.

4. **POST /v1/api/assessment/submit** — uses `@RequestPart` for multipart: the JSON `request` part is deserialized (again polymorphically via `@JsonTypeInfo`) and the `files` part carries `MultipartFile[]`. Files are wired into the DTO before factory delegation.

5. **AssignmentStrategy.submitAssessment()** — validates file count against `noOfDocumentsToBeUploaded`, checks for existing submission (idempotency), creates a `Submission`, iterates files to build `AssignmentAttachment` entities (UUID-keyed, BLOB-stored, URI-generated), then bulk-saves via `saveAll()`.

6. **QuizStrategy.submitAssessment()** — creates a `Submission`, maps each `StudentQuestionAndAnswerDTO` to a `StudentQuizQuestionResponse` (setting `isCorrectOptionChosen` from the option's stored boolean), bulk-saves, then immediately calls `ResultService.computeQuizResult()` to auto-grade.

3.5 Video Upload Flow

1. **POST /v1/api/course/add-video** — receives `MultipartFile`, `title`, `courseId`. `CourseVideoInterfaceImpl.uploadVideo()` is invoked with teacher's `userId` extracted from the authentication principal.

2. **Authorization Check** — verifies `course.getTeacher().getUserId().equals(teacherId)`. Throws `UserIdDoNothMatchException` if the authenticated user is not the course owner.

3 **Content-Type Routing** — `MediaType.parseMediaType(file.getContentType())` determines the storage subfolder: `video/`, `audio/`, or `pdf/`. Unsupported types throw `FileException`.

4 **Duration Extraction** — a temporary copy of the file is written to disk and passed to `VideoUtil.getVideoDuration()` (backed by JAVE / FFmpeg). The temp file is cleaned up in a `finally` block regardless of success or failure.

5 **Persistence** — the UUID-named file is saved to the permanent upload directory. A `CourseModule` entity is persisted, and the parent `Course.duration` is incremented. If the DB save fails, the file is deleted from disk before the exception propagates.

3.6 Exception Handling Flow

All unhandled exceptions propagate to `GlobalExceptionHandler` (`@ControllerAdvice`). The handler maps specific exception types to HTTP status codes:

Exception	HTTP Status	Notes
`UserNotFoundException`	404 NOT_FOUND	Checked; thrown by service impls
`CourseNotFoundException`	404 NOT_FOUND	Unchecked RuntimeException
`UserIdDoNothMatchException`	404 (should be 403)	Bug: wrong status code for auth failure
`ComplianceRecordNotFoundException`	404 NOT_FOUND	Returns Map body instead of DTO
`EncoderException`	500 INTERNAL_SERVER_ERROR	FFmpeg video processing failure
`IOException`	500 INTERNAL_SERVER_ERROR	File system errors
`Exception` (catch-all)	400 BAD_REQUEST	Overly broad; masks unexpected errors as client errors

4 API Endpoints Reference

Authentication

Method	Path	Auth	Description
POST	/v1/auth/register	Public	Register any role; factory + strategy selects implementation
POST	/v1/auth/login	Public	Returns JWT access token + refresh token
POST	/v1/auth/refresh	Public	Exchange refresh token for new access token

Student Management

Method	Path	Auth	Description
GET	/v1/api/student	STUDENT/ADMIN	List all students
GET	/v1/api/student/{id}	STUDENT/ADMIN	Get student by UUID
POST	/v1/api/student/{id}/update	STUDENT/ADMIN	Partial update; validates own ID matches principal
POST	/v1/api/student/{id}/delete	STUDENT/ADMIN	Soft or hard delete

Course Management

Method	Path	Auth	Description
POST	/v1/api/course/add-course	Public*	Teacher creates course; teacher extracted from principal
GET	/v1/api/course/get-course/{id}	Public*	Get course details with teacher info
POST	/v1/api/course/enrollment/{courseId}/student/{studentId}	Public*	Enroll student; checks existing enrollment
POST	/v1/api/course/add-video	Public*	Upload video/audio/PDF; validates teacher ownership
GET	/v1/api/course/stream/{moduleId}	Public*	Stream video/audio/PDF

			as resource
DEL	`/v1/api/course/{courseId}/video/{videoId}/delete-video`	Public*	Delete video; removes file from disk and DB
POST	`/v1/api/course/{courseId}/module/{moduleId}/mark-as-complete`	Public*	Updates enrollment progress and remaining duration

* Course endpoints are currently permit-all in SecurityConfig but internally verify ownership via teacher ID comparison.

Assessment

Method	Path	Auth	Description
POST	`/v1/api/assessment/create`	TEACHER	Polymorphic create; `@JsonTypeInfo` on assessmentType
POST	`/v1/api/assessment/submit`	STUDENT/ADMIN	Multipart; JSON part + files part; type-routed

Attendance, Audit, Documents, Results

Method	Path	Auth	Description
POST	`/v1/api/attendance`	STUDENT/ADMIN	Mark present; idempotent for same-day records
GET	`/v1/api/attendance/of`	STUDENT/ADMIN	Filter by studentId, courseId, or both
GET	`/v1/api/audits/{userId}`	ADMIN	Full audit history for user
GET	`/v1/api/audits/{userId}/from`	ADMIN	Paginated audit log (page size fixed at 5)

POST	/v1/api/doc/upload	STUDENT/ADMIN	Store document BLOB; returns URI
GET	/v1/api/doc/view/{uuid}	STUDENT/ADMIN	Stream document with inferred Content-Type
GET	/v1/api/result/{assessmentId}	TEACHER	Get result by assessment ID
POST	/v1/api/result/{assessmentId}/student/{studentId}/evaluate	TEACHER	Manual grading; validates teacher-course ownership

5 Security & Middleware Configuration

JWT Strategy

The application uses a **stateless JWT-based authentication** mechanism. The access token is a signed **HS256 (HMAC-SHA256)** JWT with a 10-minute TTL. The secret key is generated dynamically via `KeyGenerator.getInstance("HmacSHA256")` in `JWTService`'s constructor — meaning every application restart invalidates all issued tokens. In a production environment this key must be externalized.

Refresh Token Mechanism

A long-lived (24-hour) **opaque refresh token** is stored server-side in the `refresh_token` table. This is a hybrid approach: short-lived JWTs limit the exposure window of a stolen token, while the refresh token allows seamless re-issuance. On every `POST /v1/auth/login` the service upserts the refresh token record (same DB row, new token value), implementing **refresh token rotation**.

Security Filter Chain

```
HTTP Request
   ↓
JwtFilter (OncePerRequestFilter)
   ↓ extracts + validates Bearer token
   ↓ sets Authentication in SecurityContextHolder
   ↓
Spring Security Authorization Filter
   ↓ evaluates requestMatchers() rules in SecurityConfig
   ↓ throws AccessDeniedException → CustomAccessDeniedHandler (JSON 403)
   ↓
DispatcherServlet → Controller
```

Role-Based Access Matrix

Path Pattern	Permitted Roles
`/v1/auth/**`	Public (no auth)
`/v1/api/course/**`	Public (auth enforced internally by teacher ID check)
`/v1/api/student/**, /attendance/**, /doc/**, /assessment/submit`	STUDENT, ADMIN
`/v1/api/result/**, /teachers/**`	TEACHER
`/v1/api/parent/**`	PARENT
`/v1/api/assessment/create`	TEACHER

`/v1/api/audits/**, /compliance/**`	ADMIN
`/swagger-ui/**, /v3/api-docs/**`	Public

> **Security Gap:** Authorization is enforced only at the HTTP routing layer in `SecurityConfig`. Service methods have no `@PreAuthorize` annotations, so any internal call (scheduled task, event listener, or future messaging consumer) would bypass role checks entirely.

Password Encoding

BCrypt with cost factor 12 (`BCryptPasswordEncoder(12)`). At cost 12 each hash computation takes ~250–500ms on modern hardware, making brute-force attacks computationally expensive. Passwords are encoded in strategy implementations before entity persistence and are never returned in any response DTO.

6 Design Patterns in Use

Strategy Pattern

The **Strategy pattern** defines a family of algorithms, encapsulates each, and makes them interchangeable. In EduConnect, `UserAuthStrategy` is the strategy interface with `supports(String role)` as the discriminator and `save(User)` as the algorithm. Concrete strategies (`AdminAuthStrategy` , `StudentAuthStrategy` , `TeacherAuthStategy` , `ParentAuthStrategy`) each handle one role. The same pattern applies to `AssessmentStrategy` → `AssignmentStrategy` + `QuizStrategy` . This eliminates large if-else or switch-case blocks and makes adding a new role or assessment type **Open/Closed Principle** compliant — extend without modifying existing code.

Factory Pattern

`UserFactory` and `AssessmentFactory` are **Factory classes** that abstract the strategy selection logic. Controllers call `userFactory.executeSave(user)` or `assessmentFactory.createAssessment(teacher, dto)` without knowing which strategy runs. Spring injects all `UserAuthStrategy` implementations via `List<UserAuthStrategy>` constructor injection, enabling automatic registration of new strategies without modifying the factory.

Template Method Pattern

`UserAuthStrategy` provides a `default verify()` method containing the complete JWT + refresh token issuance pipeline (Spring authentication → JWT generation → refresh token creation → DTO assembly). Subclasses inherit this flow for free; they only override `save()` to customize persistence. This is the **Template Method pattern** implemented via a Java 8+ interface default method instead of an abstract class.

Chain of Responsibility Pattern

`UserValidator<T>` is a **functional interface** extending `Function<T, Optional<String>>` . The `and(UserValidator<T> other)` default method composes validators into a chain: if the current validator returns a non-empty error, the chain short-circuits and returns that error; otherwise it passes to the next validator. The chain is assembled in `UserFactory.executeSave()` as `EMAIL_VALID.and(PASSWORD_LENGTH).and(PASSWORD_STRENGTH)` .

Builder Pattern

Lombok's `@Builder` and `@SuperBuilder` are used extensively on entity classes. `@SuperBuilder` on `User` and its subclasses enables the builder pattern to propagate through the inheritance hierarchy, allowing code like `Student.builder().fullName(...).email(...).role(...).build()` .

Repository Pattern

All database access is abstracted behind Spring Data JPA repository interfaces. Controllers and services never call `EntityManager` directly. This decouples business logic from persistence technology — a fundamental aspect of the **Repository pattern** as described in Domain-Driven Design.

7 Suggested Improvements

CRITICAL **Externalize JWT Secret**

The secret key in `JWTService` is re-generated on every startup. Move it to `application-secret.properties` or a secrets manager. Without this fix, every server restart forces all users to re-login, and horizontal scaling is impossible (each instance signs with a different key).

CRITICAL **Migrate BLOB Storage to Object Storage**

Remove `@Lob LONGBLOB` columns from `StudentDocument` and `Attachment`. Store files on S3/MinIO and keep only the URI in the database. This eliminates the BLOB-as-bottleneck issue, enables CDN delivery for video modules, and reduces database backup size dramatically.

HIGH **Fix the AdminServiceImpl.update() Bug**

The current code calls `admin1.builder()...build()` and discards the result. The updated admin is never persisted. Fix: use the `UpdateUtil.setIfPresent()` pattern followed by `adminRepo.save(admin1)`.

HIGH **Add Method-Level Security (@PreAuthorize)**

Annotate service methods with `@PreAuthorize("hasRole('TEACHER')")`. This ensures authorization is enforced regardless of how a service method is invoked (HTTP, internal, scheduled, or event-driven), following the **Defense in Depth** principle.

HIGH **Implement the Notification System**

The `Notification` entity and `NotificationType` enum exist but no sending logic is wired. Use Spring's `ApplicationEventPublisher` to publish domain events (e.g., `AssessmentSubmittedEvent`) and a listener to persist `Notification` records. This decouples cross-domain concerns without tight service coupling.

MEDIUM **Paginate All List Endpoints**

`StudentService.getAll()` and `AdminServiceImpl.getAll()` perform unbounded `findAll()` queries. Introduce `Pageable` parameters as already done in `TeacherService.getAll(Pageable)`. This prevents OOM errors as data grows.

MEDIUM **Fix GlobalExceptionHandler Status Codes**

`UserIdDoNothMatchException` is mapped to 404 NOT_FOUND when it should be 403 FORBIDDEN. The catch-all `Exception.class` handler maps all unexpected errors to 400 BAD_REQUEST — it should be 500

INTERNAL_SERVER_ERROR to correctly signal server-side failures to clients.

8 Microservices Migration Plan

The EduConnect monolith is a good candidate for decomposition because its domains are already well-separated by package structure and service interfaces. A **Strangler Fig pattern** migration is recommended — extract one bounded context at a time while keeping the monolith running.

Bounded Context Decomposition

Service	Owns	Database	Priority
Identity Service	User, Auth, Roles, JWT, RefreshToken	users DB	Extract first
Academic Service	Course, Module, Assessment, Quiz, Assignment, Submission, Enrollment, Result, Attendance	academic DB	Extract second
Document Service	StudentDocument, Attachment, DocType	documents DB + S3	Strangler-fig alongside BLOB migration
Compliance & Audit Service	AuditLog, ComplianceRecord, Note, Report	audit DB	Extract independently
Notification Service	Notification, EmailService, ParentVerificationToken	notifications DB	New service, event-driven
Demographics Service	StudentDemographics, ParentAccess	profile DB	Low risk; limited cross-service calls

Infrastructure Requirements

API Gateway (Spring Cloud Gateway or Kong) — handles JWT validation at the edge, routes by path prefix, provides rate limiting and circuit breaking via Resilience4j.

Service Registry (Eureka or Consul) — each service registers on startup; the gateway resolves downstream URLs dynamically.

Event Bus (Kafka or RabbitMQ) — replaces synchronous cross-service calls with asynchronous events. Example: `AssessmentSubmitted` event published by Academic Service, consumed by Compliance Service (record creation) and Notification Service (push message).

Config Server (Spring Cloud Config) — externalizes `application.properties` per service, including the JWT secret shared across Identity and all verifying services.

Distributed Tracing (Micrometer + Zipkin) — replaces the current logback file-per-level approach with correlated trace IDs across service hops.

Data Migration Challenge: Shared Tables

The primary challenge is the `users` table being accessed by every domain. The **recommended approach** is: (1) each service validates JWTs locally using the shared secret (no round-trip to Identity Service per request); (2) services store only the `userId`

UUID as a foreign reference, never JOIN to user data from another service's database; (3) user profile data needed by Academic Service (e.g., `fullName`) is replicated via a `UserProfileUpdated` event rather than a live cross-service query.

9 Interview Q&A — Core Java

QUESTION

What is the difference between `Optional.of()` and `Optional.ofNullable()`? Where is this misused in EduConnect?

ANSWER

`Optional.of(value)` throws a `NullPointerException` immediately if `value` is null. It should only be used when you are certain the value is non-null. `Optional.ofNullable(value)` safely wraps a potentially-null value, returning `Optional.empty()` if null. In `CourseVideoImpl.uploadVideo()`, the code `Optional.of(file).filter(f → f.getContentType().startsWith("video/"))` would throw a `NullPointerException` if `getContentType()` returns null (which is possible for multipart uploads with no declared content type). The correct call is `Optional.ofNullable(file.getContentType()).filter(ct → ct.startsWith("video/"))`.

QUESTION

What is the difference between checked and unchecked exceptions? How does EduConnect mix them, and what problem does this cause?

ANSWER

Checked exceptions extend `Exception` and must be declared in the method signature (`throws` clause) or handled in a `try-catch`. They represent recoverable conditions the caller should handle (e.g., `UserNotFoundException`). **Unchecked exceptions** extend `RuntimeException` and do not require declaration. EduConnect's service implementations frequently wrap checked exceptions in RuntimeException: `catch (UserNotFoundException e) { throw new RuntimeException(e); }`. This causes the `GlobalExceptionHandler` to see a `RuntimeException` wrapper instead of the original type, falling through to the catch-all handler and returning a generic 400 instead of the intended 404.

QUESTION

What is the significance of `@SuperBuilder` vs `@Builder` in Lombok?

ANSWER

`@Builder` generates a builder for the annotated class only and does not include fields from superclasses. In a JPA inheritance hierarchy like `User → Student`, a `@Builder` on `Student` would not expose `User`'s fields (email, password, fullName). `@SuperBuilder` generates a builder that propagates through the entire inheritance chain, so `Student.builder().email(...).password(...).enrollmentNumber(...).build()` works correctly. Both parent and child classes must be annotated with `@SuperBuilder` for this to work.

QUESTION

Explain the Java functional interface used in `UserValidator`. What is a `Function<T, R>`?

ANSWER

A **functional interface** is an interface with exactly one abstract method, making it eligible for lambda expressions and method references. `Function<T, R>` is a built-in functional interface representing a function that accepts one argument of type T and returns a result of type R. `UserValidator<T>` extends `Function<T, Optional<String>>`, meaning each validator takes a user entity and returns either an error message or empty. The `and()` default method leverages this to compose validators without modifying any existing code — a textbook application of the **Open/Closed Principle** and **functional composition**.

QUESTION

What is the difference between `==` and `.equals()` for UUID comparison? Is there a bug in EduConnect?

ANSWER

`==` on objects compares **reference identity** (same memory address). `.equals()` compares **value equality**. Two `UUID` objects representing the same UUID value will have `uuid1.equals(uuid2) == true` but `uuid1 == uuid2` is false if they are different object instances. In `AssignmentStrategy.canCreateAssessment()` and `ResultServiceImpl.evaluateStudent()`, the comparison `teacher.getUserId() == course.getTeacher().getUserId()` uses `==` on `UUID` objects. This is incorrect and will evaluate to `false` even when the UUIDs are equal, because each `getUserId()` call returns a distinct object. The correct form is `teacher.getUserId().equals(course.getTeacher().getUserId())`.

QUESTION

What does `@Lob` with `@Basic(fetch = FetchType.LAZY)` do in JPA, and why is it important here?

ANSWER

`@Lob` maps a Java field to a database Large Object (BLOB or CLOB). `@Basic(fetch = FetchType.LAZY)` instructs Hibernate to **defer loading** of the field until it is explicitly accessed on the entity instance. Without `LAZY`, every query loading a `StudentDocument` entity (e.g., listing documents by student) would pull the entire file binary into memory. With `LAZY`, the BLOB is only loaded when `document.getFileData()` is called. This is critical performance mitigation for BLOB-in-DB storage patterns. Note: `LAZY` loading of basic fields requires Hibernate bytecode enhancement or instrumentation to work reliably; without it Hibernate may silently fall back to eager loading.

10 Interview Q&A — Collections & Streams

QUESTION

Rewrite the quiz result computation loop in `ResultServiceImpl` using Java Streams.

ANSWER

```
long total = studentQuizQuestionResponseList.size();
long correct = studentQuizQuestionResponseList.stream()
    .filter(r -> Boolean.TRUE.equals(r.getIsCorrectOptionChosen()))
    .count();
double score = total == 0 ? 0.0 : (double) correct / total;
```

Using `Boolean.TRUE.equals()` instead of a direct unboxing `r.getIsCorrectOptionChosen()` avoids a `NullPointerException` if the field is null, which is a real risk given the entity lacks a `@Column(nullable=false)` constraint on that field.

QUESTION

What is the difference between `findFirst()` and `findAny()` in Java Streams? Which should be used in the factory strategy selection?

ANSWER

`findFirst()` always returns the first element of the stream in encounter order and is **deterministic** even in parallel streams. `findAny()` returns any element and may return a different element on each execution, which is an optimization hint for parallel streams. In the strategy selection code, `.stream().filter(...).findFirst()` is correct because the list of strategies has a defined order (Spring's bean discovery order) and we want deterministic behavior. `findAny()` would introduce non-determinism if the stream were ever made parallel.

QUESTION

When would you use `HashMap` vs `LinkedHashMap` vs `TreeMap`?

ANSWER

`HashMap` offers O(1) average-case put/get with no ordering guarantee — used in `StudentDocumentServiceImpl.allowedTypes` for file extension to enum mapping. `LinkedHashMap` maintains **insertion order**, making it suitable for LRU caches or ordered API responses. `TreeMap` maintains **natural sorted order** (or a custom `Comparator`) with O(log n) operations, useful when sorted iteration is required (e.g., a leaderboard by score). In EduConnect, the `allowedTypes` map in `StudentDocumentServiceImpl` is correctly a `HashMap` since lookup performance matters and order is irrelevant.

QUESTION

What is the difference between `.toList()` (Java 16+) and `Collectors.toList()`?

ANSWER

`Stream.toList()` (introduced in Java 16) returns an **unmodifiable list** — attempts to add or remove elements throw `UnsupportedOperationException`. `Collectors.toList()` returns a modifiable `ArrayList` (though the spec doesn't guarantee the implementation). EduConnect uses `.toList()` in several service implementations — this is correct when the result is read-only, but would fail at runtime if the caller attempts to mutate it (e.g., `student.getEnrollments().add(e)` after assigning from a `toList()` result). In `CourseServiceImpl.addStudentToCourse()`, the code correctly checks for null and initializes a new `ArrayList` before adding.

QUESTION

Explain the difference between `Comparator.comparing()` and implementing `Comparable`.

ANSWER

`Comparable<T>` is implemented on the class itself (`compareTo()` method) and defines the **natural ordering** — there is exactly one. `Comparator<T>` is an external comparator object that defines a **custom ordering** independent of the class. `Comparator.comparing()` is a static factory that creates a comparator from a key-extraction function, composable via `.thenComparing()`. In EduConnect, if you wanted to sort assessments by due date then by title, you'd use `Comparator.comparing(Assessment::getDueDate).thenComparing(Assessment::getTitle)` rather than modifying the `Assessment` class.

11 Interview Q&A — Design Patterns

QUESTION

Explain the Strategy pattern with the exact code example from EduConnect.

ANSWER

The Strategy pattern defines a family of algorithms (strategies), encapsulates each in its own class, and makes them interchangeable at runtime. The **context** is `UserFactory`; the **strategy interface** is `UserAuthStrategy` with `supports(String role)` and `save(User)`; the **concrete strategies** are `AdminAuthStrategy`, `StudentAuthStrategy`, `TeacherAuthStategy`, and `ParentAuthStrategy`. At runtime, `UserFactory.executeSave()` calls `strategyList.stream().filter(s → s.supports(role)).findFirst()` to select the correct algorithm. The key benefit: adding a new role (e.g., LIBRARIAN) requires only a new class implementing `UserAuthStrategy` annotated with `@Service` — zero changes to `UserFactory`. This satisfies the **Open/Closed Principle**.

QUESTION

What is the difference between Factory Method and Abstract Factory design patterns?

ANSWER

The **Factory Method** pattern defines an interface for creating an object but lets subclasses decide which class to instantiate. It relies on inheritance. The **Abstract Factory** pattern provides an interface for creating *families of related objects* without specifying concrete classes. It relies on composition. EduConnect's `UserFactory` is closer to a **Service Locator / Strategy Dispatcher** — it's often casually called a "factory" but technically selects among pre-existing strategy beans rather than instantiating new objects. A true Factory Method would be each `UserAuthStrategy.save()` method acting as the factory for the entity type (which it does — `AdminAuthStrategy.save()` creates and returns an `Admin`).

QUESTION

Explain the Builder pattern and why `@SuperBuilder` is needed in a JPA inheritance hierarchy.

ANSWER

The **Builder pattern** separates the construction of a complex object from its representation, allowing the same construction process to produce different representations. It solves the "telescoping constructor" problem. In JPA inheritance (`User → Student`), Lombok's `@Builder` generates a builder only for the annotated class's own fields, excluding the parent's. `@SuperBuilder` generates a builder hierarchy that mirrors the class hierarchy — the `StudentBuilder` extends `UserBuilder`, so a single builder call can set both `User.email` and `Student.enrollmentNumber`. Both superclass and subclass must declare `@SuperBuilder`; mixing `@Builder` on the parent with `@SuperBuilder` on the child is unsupported and generates a compile error.

QUESTION

What is the Singleton pattern? How does Spring Boot implement it, and what is the EduConnect implication?

ANSWER

The **Singleton pattern** ensures a class has exactly one instance and provides a global point of access to it. Spring Boot implements Singleton scope by default for all `@Component`, `@Service`, `@Repository`, and `@Controller` beans — the Spring IoC container creates one instance and injects the same reference everywhere. In EduConnect, this means `JWTService` (with its in-memory generated secret key) is a singleton — all controllers and filters share the same key instance. This is why server restarts invalidate all tokens: the singleton is destroyed and recreated with a new key. The implication for EduConnect is that the key must be stored outside the bean's state (e.g., in a properties file) to survive restarts.

QUESTION

What is the Observer pattern and how would it benefit EduConnect's notification system?

ANSWER

The **Observer pattern** (also called Publish-Subscribe) defines a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified automatically. In EduConnect, the `Notification` entity exists but nothing publishes or consumes notification events. Using Spring's `ApplicationEventPublisher`, a service like `AssessmentServiceImpl` could publish an `AssessmentSubmittedEvent` (the subject), and multiple independent listeners (notification persistence, email dispatch, compliance record creation, audit logging) could react without the Assessment service knowing they exist. This achieves **loose coupling** between domains, the core benefit of the Observer pattern.

12 Interview Q&A — Spring Boot & JPA

QUESTION

What is the difference between Spring's `@Transactional` and Jakarta's `@Transactional`? Which should be used in Spring Boot?

ANSWER

Spring's `org.springframework.transaction.annotation.@Transactional` integrates with Spring's **PlatformTransactionManager** abstraction and supports additional attributes: `readOnly`, `propagation`, `isolation`, `timeout`, and `rollbackFor`. Jakarta's `jakarta.transaction.@Transactional` is a JTA annotation originally designed for JEE application servers. In a Spring Boot application with JPA, Spring's annotation is always preferred. In EduConnect, `AttendanceServiceImpl` incorrectly uses Jakarta's annotation while `StudentServiceImpl` correctly uses Spring's with `readOnly=true` on query methods — a performance optimization that prevents Hibernate from tracking entity state (dirty checking) for read-only operations.

QUESTION

Explain JPA's N+1 query problem and identify where it might occur in EduConnect.

ANSWER

The **N+1 problem** occurs when loading a collection of N entities each triggers an additional query to load a related entity, resulting in N+1 total queries instead of 1. In EduConnect, `CourseServiceImpl.getAllCourse()` calls `courseRepo.findAll()` and then `courseMapper.toResponseDTO(course)` for each course. The mapper accesses `course.getTeacher().getUserId()` — if `Teacher` is `FetchType.LAZY`, each call triggers a separate SELECT to load the teacher, producing N+1 queries for N courses. The fix is a JPQL fetch join: `@Query("SELECT c FROM Course c JOIN FETCH c.teacher")` in `CourseRepo`.

QUESTION

What does `@JsonTypeInfo` with `visible = true` do on `CreateAssessmentRequestDTO`?

ANSWER

`@JsonTypeInfo` instructs Jackson to use a field in the JSON payload as a **type discriminator** to determine which concrete subclass to deserialize into. The property `assessmentType` in the JSON body selects either `CreateAssignmentRequestDTO` or `CreateQuizRequestDTO` via the `@JsonSubTypes` annotation. `visible = true` means the discriminator field is **retained** in the deserialized object's state — without it, Jackson would consume the field during type resolution and the resulting object would have `assessmentType == null`, breaking all downstream code that checks the type. This is a polymorphic deserialization pattern — a powerful but easily misconfigured Jackson feature.

QUESTION

What are Spring Data JPA derived query methods? Give examples from this codebase.

ANSWER

Derived query methods are repository interface methods whose SQL is automatically generated by Spring Data JPA from the method name, following a naming convention. Examples from EduConnect: `findByStudentUserId(UUID)` in `AttendanceRepo` generates `SELECT a FROM Attendance a WHERE a.student.userId = ?1`; `existsByStudentUserIdAndCourseCourseId(UUID, UUID)` in `EnrollmentRepo` generates an existence check with two conditions; `findByEmail(String)` in `UserRepo` generates a simple equality lookup. When queries become too complex for derivation (e.g., multi-table joins with conditions), `@Query` with JPQL (as in `AttendanceRepo.findByCourseAndAttendance()`) or native SQL is used.

QUESTION

What is `@ControllerAdvice`? How does `GlobalExceptionHandler` work?

ANSWER

`@ControllerAdvice` is a specialization of `@Component` that allows handling exceptions across the whole application globally, rather than within individual controllers. It applies `@ExceptionHandler`, `@InitBinder`, and `@ModelAttribute` to all controllers. `GlobalExceptionHandler` in EduConnect maps specific exception types to HTTP status codes: `UserNotFoundException` → 404, `EncoderException` → 500, etc. Spring dispatches to the most specific handler first. The catch-all `Exception.class` handler fires last for any unhandled exception, wrapping the error in an `ErrorResponseDTO` with timestamp, status, and message fields — providing a consistent error contract for API consumers.

QUESTION

What is Bean Validation and how is it used in EduConnect?

ANSWER

Bean Validation (Jakarta Validation, formerly JSR-380) provides annotations (`@NotNull`, `@NotBlank`, `@Email`, `@Min`, etc.) that declare constraints on object fields. Spring Boot auto-configures a `Validator` bean; `@Valid` or `@Validated` on a controller parameter triggers validation before the method executes, throwing `MethodArgumentNotValidException` on failure. In EduConnect, `DemographicsRequestDTO` uses `@NotBlank`, `@NotNull`, and `@Email` constraints, and the controller method annotates it with `@Valid`. However, the `GlobalExceptionHandler` does not have a specific handler for `MethodArgumentNotValidException`, so validation failures fall through to the generic 400 handler without field-specific error messages.

13 Interview Q&A — System Design & Architecture

QUESTION

What is the difference between authentication and authorization? How does EduConnect implement each?

ANSWER

Authentication is the process of verifying *who* a user is (identity verification). In EduConnect, this is handled by `JwtFilter` + `EduconnectUserDetailsService`: the JWT is validated, the user is loaded from the database, and an `Authentication` object is placed in the `SecurityContextHolder`. **Authorization** is the process of verifying *what* an authenticated user is allowed to do. In EduConnect, this is handled by Spring Security's `HttpSecurity.authorizeHttpRequests()` configuration in `SecurityConfig`, which maps URL patterns to required roles. The key distinction: authentication happens once (in the filter); authorization happens per-endpoint (in the security filter chain).

QUESTION

What is a stateless vs stateful REST API? Why is EduConnect stateless and what are the trade-offs?

ANSWER

A **stateful** API maintains server-side session state between requests (e.g., HTTP sessions stored in memory or a session store). A **stateless** API requires each request to carry all necessary information (e.g., a JWT containing user identity and role). EduConnect is stateless: `SessionCreationPolicy.STATELESS` in `SecurityConfig` prevents Spring Security from creating HTTP sessions. **Advantages**: horizontal scalability (any server instance can handle any request since no session affinity is needed), simpler server design, and better REST compliance. **Disadvantages**: tokens cannot be invalidated server-side before expiry (a stolen 10-minute JWT is valid for the full 10 minutes), and token payload increases request size compared to a session cookie.

QUESTION

What is SOLID? Identify one violation and one correct application in EduConnect.

ANSWER

SOLID: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. **Correct application (Open/Closed)**: adding a new role to the system requires only a new `UserAuthStrategy` implementation — no modification to `UserFactory` or existing strategies. The factory is closed for modification but open for extension. **Violation (Single Responsibility)**: `StudentDocumentServiceImpl.saveStudentDocument()` handles file type detection, DocType lookup-and-create, BLOB writing, URI construction, and persistence — multiple responsibilities in one method. Extracting `DocTypeResolver` and `UriGenerator` collaborators would improve cohesion. **Violation (Interface Segregation)**: `CourseVideoService` bundles upload, streaming, deletion, update, and progress-tracking into one interface — clients that only need streaming are forced to depend on the upload contract.

QUESTION

What is idempotency in REST APIs, and where does EduConnect implement it?

ANSWER

Idempotency means that making the same request multiple times produces the same result as making it once — repeated calls do not cause unintended side effects. In EduConnect, attendance marking is idempotent: `AttendanceServiceImpl.addAttendance()` calls `attendanceRepo.existsByStudentAndCourseAndDate(s, c, LocalDate.now())` before creating a record and returns `true` without creating a duplicate if one already exists. Similarly, submission is guarded by `submissionRepo.existsByStudentAndAssessment()` and a `UNIQUE` database constraint — a second submission attempt throws an exception rather than creating a duplicate record. HTTP `GET`, `PUT`, and `DELETE` are expected to be idempotent by convention; `POST` is not (though EduConnect's attendance POST is).

QUESTION

What is connection pooling and why does it matter for a Spring Boot app like EduConnect?

ANSWER

Connection pooling maintains a pool of pre-established database connections that are reused across requests, avoiding the overhead of creating a new TCP connection and MySQL handshake for every query. Spring Boot auto-configures **HikariCP** (the default connection pool) when spring-data-jpa is on the classpath. Without pooling, a web application under load would exhaust the database's connection limit. In EduConnect, the BLOB upload/stream endpoints are particularly risky — a large file upload holds a connection for the duration of the transaction. HikariCP's default pool size is 10; under concurrent uploads this could starve the pool. Configuring `spring.datasource.hikari.maximum-pool-size` and moving BLOBs out of the database (see improvement section) are the correct mitigations.

QUESTION

What is the CAP theorem and how does it apply to a monolithic MySQL-backed system like EduConnect?

ANSWER

The **CAP theorem** states that a distributed system can guarantee at most two of three properties simultaneously: **Consistency** (every read receives the most recent write), **Availability** (every request receives a response), and **Partition Tolerance** (the system continues operating despite network partitions). A monolithic Spring Boot app with a single MySQL primary is a **CA system** by nature: it prioritizes consistency (ACID transactions) and availability within a single data center but does not provide partition tolerance (there is no distributed component). When EduConnect migrates to microservices, each service must choose its position on the CAP triangle — an Academic Service might choose CP (strong consistency for grade records) while a Notification Service might choose AP (eventual consistency is acceptable for notifications).

QUESTION

What is caching and where would you introduce it in EduConnect?

ANSWER

Caching stores the result of an expensive operation in fast storage (memory) to avoid recomputing it on subsequent requests. Spring Boot supports declarative caching via `@Cacheable`, `@CacheEvict`, and `@CachePut` with backends like Redis or Caffeine. In EduConnect, high-value cache targets are: (1) `CourseService.getAllCourse()` — the course catalog changes rarely but is read frequently; (2) `TeacherService.getById()` — teacher profiles are stable; (3) JWT public key / signing key (if externalized) — can be cached with a short TTL. Cache invalidation strategy: evict course cache on `addCourse()` or `deleteById()`. The `StudentDocument` BLOB endpoint would benefit from **HTTP cache headers** (`Cache-Control: public, max-age=86400`) already set in the controller, allowing CDN or browser caching of served documents.

To save as PDF: use your browser's **Print** → **Save as PDF** function. Set margins to Minimum for best results.